Solo-Git v1.0 Feature Lock

This document defines the complete feature set for Solo-Git v1.0 release. Features not listed here are deferred to future versions.

Release Date: November 2024
Version: 1.0.0
Status: Feature Lock - No new features until v1.1

Core Philosophy

Solo-Git is a **Git workflow manager** that eliminates branching complexity through:
1. **Workpads** instead of branches
2. **AI-assisted development** with Abacus-first routing
3. **Test-driven workflow** with automatic validation
4. **Fast-forward only** - no merge conflicts

✅ v1.0 Features

1. Repository Management

Repository Initialization

- `sologit init` - Initialize repository from scratch
- `sologit init --from-zip <file>` - Import from ZIP archive
- `sologit init --from-git <url>` - Clone from Git URL
- Automatic `.sologit/` state directory creation
- Git repository initialization if not exists

State Management

- JSON-based state file (`.sologit/state.json`)
- Bidirectional Git ↔ State synchronization
- Automatic state recovery on corruption
- State backup on every modification
- Workpad metadata tracking

File System Operations

- Workpad-specific working directories
- Automatic `.gitignore` generation
- File change detection and tracking
- Diff generation for all workpads

---

2. Workpad System (No Branches!)

Workpad Lifecycle

- **Create**: `sologit workpad create <id> <title>` - New feature workpad
- **List**: `sologit workpad list` - Show all workpads with status
- **Info**: `sologit workpad info <id>` - Detailed workpad information
- **Delete**: `sologit workpad delete <id>` - Remove workpad (safe)

Checkpointing

- **Checkpoint**: `sologit workpad checkpoint <id> <message>` - Save progress
- **History**: `sologit workpad history <id>` - View checkpoint history
- Automatic timestamp tracking
- File snapshot on each checkpoint
- Revert to any checkpoint

Promotion (Replace Merging)

- **Promote**: `sologit workpad promote <id>` - Fast-forward to main
- Automatic promotion rules validation
- Test execution before promotion
- Rollback on failure
- No merge conflicts ever!

Workpad Status

- Active/completed status tracking
- File change summary
- Test result integration
- AI cost tracking per workpad

---

3. AI Routing System (Abacus-First)

Provider Architecture

- **Primary**: Abacus.AI with RouteLLM
- **Fallback #1**: OpenAI (gpt-4o-mini)
- **Fallback #2**: Anthropic (claude-3-5-sonnet)
- Automatic failover on errors
- Health checking with caching
- Unified provider interface

Routing Strategies

- **Abacus-First** (default) - Abacus → OpenAI → Anthropic
- **Cost-Optimized** - Cheapest available first
- **Latency-Optimized** - Fastest response first
- **User-Specified** - Manual provider selection

Commit Message Generation

- **Command**: `sologit commit-msg -w <workpad>`
- Analyzes diff and generates Conventional Commits format
- Supports free-form or structured messages

- Interactive editing before commit
- Metadata tracking (provider, model, cost, latency)

API Integration

- Standardized request/response format
- Automatic retry on transient failures
- Rate limit handling
- Token usage tracking
- Cost estimation per request

---

4. Test Orchestration

Test Execution

- **Fast Tests**: `sologit test fast` - Unit tests only (<30s)
- **Full Tests**: `sologit test full` - All tests including integration
- Sandboxed Docker execution
- Parallel test execution (configurable workers)
- Timeout enforcement (per-test and global)
- Real-time progress reporting

Test Frameworks Supported

- pytest (Python)
- unittest (Python)
- jest (JavaScript/TypeScript)
- mocha (JavaScript/TypeScript)
- Go test (Go)
- Custom framework support via config

Test Analysis

- **9-Category Failure Classification**:
  1. `ASSERTION_FAILURE` - Test logic failed
  2. `DEPENDENCY_ERROR` - Missing imports/modules
  3. `TIMEOUT` - Test exceeded time limit
  4. `SETUP_ERROR` - Fixture/setup failed
  5. `TEARDOWN_ERROR` - Cleanup failed
  6. `RESOURCE_ERROR` - Memory/disk issues
  7. `NETWORK_ERROR` - API/network failures
  8. `SYNTAX_ERROR` - Code syntax issues
  9. `UNKNOWN_ERROR` - Uncategorized failures

- Test result aggregation

- Coverage reporting integration
- Failure pattern detection
- Flaky test identification

---

5. Workflow Automation

Auto-Merge Pipeline

1. **Test Execution** - Run fast + full tests
2. **Failure Analysis** - Categorize failures
3. **Promotion Gate** - Check rules
4. **Auto-Promotion** - Fast-forward main
5. **CI Smoke Tests** - Post-merge validation
6. **Auto-Rollback** - Revert on CI failure

Promotion Rules (Configurable)

- `require_passing_tests`: true/false
- `fast_forward_only`: true (v1.0 only supports this)
- `max_changed_files`: int (prevent large changes)
- `min_test_coverage`: percentage
- `blocked_files`: list (e.g., `["config/prod.yaml"]`)
- `require_review`: bool (manual gate)

CI Integration

- Post-promotion smoke tests
- Deployment simulation
- Performance regression tests
- Security scanning hooks
- Custom CI script execution

Rollback Handler

- Automatic rollback on CI failure
- Workpad recreation from backup
- State restoration
- Git tree reset
- Notification system

---

6. User Interfaces

CLI (Command-Line Interface)

- **Rich formatting** - Colors, tables, progress bars
- **Interactive prompts** - User-friendly dialogs
- **Command aliases** - Short commands (e.g., `wp` for `workpad`)
- **Shell completion** - Bash/Zsh autocomplete
- **Piped output** - JSON/CSV for automation

Key Commands:

- `sologit init` - Initialize repository
- `sologit workpad *` - Workpad management
- `sologit test *` - Test execution
- `sologit commit-msg` - AI commit messages
- `sologit config *` - Configuration management

- `sologit heaven` - Launch GUI
- `sologit tui` - Launch TUI

TUI (Terminal UI)

- **Textual framework** - Keyboard-first interface
- **Panels**:
- Workpad list with status
- File tree browser
- Commit history graph (ASCII art)
- Test result viewer
- AI chat panel
- **Keyboard shortcuts**:
- `Ctrl+C` - Cancel/Quit
- `Tab` - Navigate panels
- `Enter` - Select/Open
- `?` - Help overlay

Heaven GUI (Desktop App - Tauri)

- **Workpad Management**:
- Visual workpad list with status cards
- Drag-and-drop file management
- Checkpoint timeline view
- **Code Editor**:
- Monaco Editor integration
- Syntax highlighting for 50+ languages
- Multi-tab editing
- Git diff visualization
- **Commit Graph**:
- D3.js visualization
- Interactive node navigation
- Workpad timeline overlay
- **Metrics Dashboard**:
- Recharts-based analytics
- Test success rates
- AI cost breakdown
- Commit frequency graphs
- **AI Chat Panel**:
- Context-aware code assistance
- Inline code generation
- Commit message suggestions
- **Command Palette** (Cmd/Ctrl+K):

- Fuzzy search commands
- Recent commands history
- Keyboard navigation

- **Settings Panel**:

- API key management
- Budget configuration
- Test framework selection
- Theme customization

---

7. Configuration System

Config Management

- YAML-based configuration (`.sologit/config.yaml`)
- Environment variable support
- Multi-profile support (dev, staging, prod)
- Template system for common configs
- Secure credential storage

Configuration Commands

- `sologit config init` - Create default config
- `sologit config show` - Display current config
- `sologit config set <key> <value>` - Update setting
- `sologit config get <key>` - Read setting
- `sologit config test` - Validate configuration
- `sologit config templates` - List templates

Configurable Settings

- AI provider credentials (Abacus, OpenAI, Anthropic)
- Budget limits (daily, monthly, alert thresholds)
- Test framework and execution settings
- Promotion rules
- Workflow automation toggles
- UI preferences

---

8. Cost Management

Budget Tracking

- Daily/monthly spending limits
- Alert thresholds (% of budget)
- Per-workpad cost tracking
- Per-provider cost breakdown
- Token usage monitoring

Cost Guard

- Pre-request budget checks
- Automatic request blocking on limit
- Cost estimation for complex operations
- Spending reports (CLI + GUI)

Cost Optimization

- Automatic model selection for task complexity
- Caching to reduce redundant requests
- Batch request optimization
- Provider cost comparison

---

9. Installation & Distribution

Installers

- **Windows**:
- MSI installer (Enterprise)
- NSIS installer (Standard)
- **Linux**:
- DEB package (Debian/Ubuntu)
- AppImage (Universal)

Package Distribution

- PyPI package (`pip install sologit`)
- GitHub Releases (with installers)
- Docker image (future)

⚠️ Known Limitations (v1.0)

Not Supported

1. **Branch-based workflows** - Use workpads only
2. **Merge commits** - Fast-forward only
3. **Rebase operations** - Not needed with workpads
4. **Submodules** - Not yet supported
5. **Large file storage (LFS)** - Planned for v1.1
6. **macOS installers** - Windows and Linux only
7. **Multi-repository projects** - Single repo per instance
8. **Collaborative workpads** - Single user per workpad
9. **Remote workpad sync** - Local only
10. **Custom Git hooks** - Predefined hooks only

Platform Support

- ✅ **Windows 10/11** (x64)
- ✅ **Linux** (Ubuntu 20.04+, Debian 11+)

- ❌ **macOS** (deferred to v1.2)
- ❌ **ARM** (experimental, unsupported)

🚀 Coming in Future Versions

v1.1 (Q1 2025)

- macOS installers (.dmg, .app)
- Large file support (LFS)
- Remote workpad synchronization
- GitHub integration
- GitLab integration

v1.2 (Q2 2025)

- Multi-user workpads (collaboration)
- Real-time co-editing
- Advanced code review tools
- Performance profiling integration

v2.0 (Q3 2025)

- Multi-repository projects (monorepo support)
- Custom workflow DSL
- Plugin system for extensibility
- Cloud-hosted Solo-Git service

📊 Version Comparison

Feature	v1.0	v1.1	v2.0
Workpad System	✅	✅	✅
AI Routing	✅	✅	✅
Test Orchestration	✅	✅	✅
Windows/Linux Installers	✅	✅	✅
macOS Installers	❌	✅	✅
Remote Sync	❌	✅	✅
Multi-user	❌	❌	✅
Monorepo Support	❌	❌	✅
Plugin System	❌	❌	✅

📝 Feature Lock Policy

v1.0 is feature-locked as of November 2024.

This means:
- ✅ **Bug fixes** are allowed
- ✅ **Performance improvements** are allowed
- ✅ **Documentation updates** are allowed
- ✅ **Security patches** are allowed
- ❌ **New features** are deferred to v1.1+

To request a feature for v1.1+, please:
1. Open a GitHub issue
2. Use the “Feature Request” template
3. Explain the use case
4. Vote on existing feature requests

🎯 Success Metrics

v1.0 is considered successful if:
1. **95%+ tests passing** (currently: 89.8%, improving)
2. **Windows + Linux installers** work one-click
3. **AI routing system** has <5% failure rate

4. **Documentation** is complete and clear
5. **Community feedback** is positive (>4.0★ rating)

📚 Documentation

- INSTALL.md (INSTALL.md) - Installation guide
- QUICKSTART.md (QUICKSTART.md) - Quick start tutorial
- docs/ (docs/) - Detailed documentation
- heaven-gui/BUILD_INSTALLERS.md (heaven-gui/BUILD_INSTALLERS.md) - Build guide

---

🤝 Contributing

v1.0 is feature-locked, but contributions are welcome for:
- Bug fixes
- Test improvements
- Documentation enhancements
- Installer improvements

See CONTRIBUTING.md (CONTRIBUTING.md) for guidelines.

Solo-Git v1.0 - Git workflow management, simplified. 🚀